Nina Pakshina

Go developer in an online retail company. LinkedIn: https://www.linkedin.com/in/nina-pakshina-228156114/ Buy me a coffee: https://www.buymeacoffee.com/ninucium

Follow writer

Golang + Golang Tutorial + Interview + Programming + Programming Languages +

Go Interview Questions, Part 2: Slices

Nina Pakshina Follow 9 min read · May 31, 2023

439 6

Hi, my name is Nina, and I work as a Go developer.

If you have ever interviewed for a Go developer position, you've likely encountered tasks related to slices. Usually, my brain "shuts off" when solving these tasks. However, knowing the peculiarities of slices in Go, they can be solved quite easily.

Generated with https://beta.dreamstudio.ai

. . .

Task 1: Reference Type

I was once given this task in one of the live coding interviews:

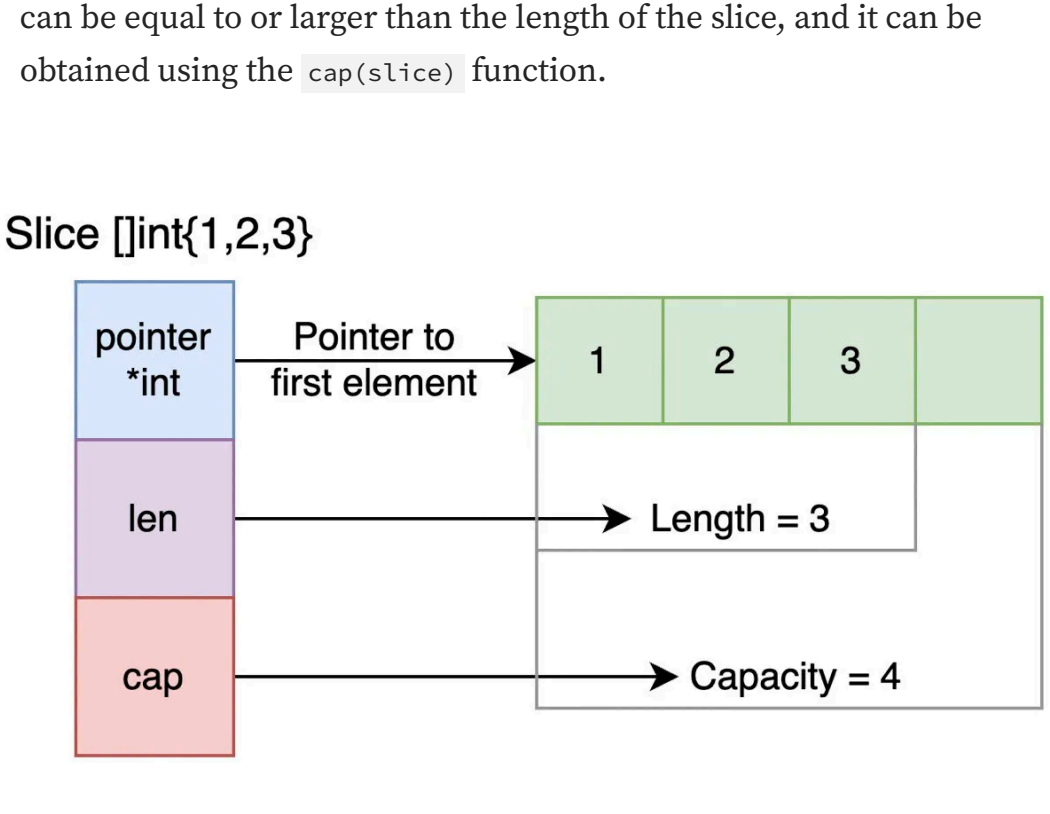
```
func main() {
  var x []int
  x = append(x, 1)
  x = append(x, 2)
  x = append(x, 3)
  y := x
  x = append(x, 4)
  y = append(y, 5)
  x[0] = 0

  fmt.Println(x)
  fmt.Println(y)
}
```

The question was: What will the lines `fmt.Println(x)` and `fmt.Println(y)` output?

Before answering, let's recall what a slice in Go represents.

A slice is a reference type that includes three main components:

1. Pointer to the array. This pointer points to the first element of the slice in the underlying array (the 'slice' type in Go is an abstraction built over the array type).
2. Length. This is the number of stored elements in the slice. The length determines the current size of the slice, and it can be obtained using the `len(slice)` function.
3. Capacity. The capacity of a slice determines how many elements it can contain without the need to expand the underlying array. The capacity can be equal to or larger than the length of the slice, and it can be obtained using the `cap(slice)` function.



Structure of slice in Go

All of these elements fully describe a slice. The pointer to the first element allows us to locate the start of the slice in memory, the length determines the number of stored elements in the slice, and the capacity allows us to manage memory, allocating new space in memory for the slice if we want to add more elements than the current capacity allows.

You can read more about slices here.

Let's return to the task and step by step look at how the values for variables x and y will change.

Step 1

Creating an empty slice:

```
var x []int
```

In Go, when creating a slice using the syntax `var x []int`, no memory is allocated for the slice. It means that the pointer of slice will have a value of `nil` (an empty pointer), indicating there's no reference to any underlying array.

This slice will have zero length and capacity. Therefore, if we print result with `fmt.Printf("x=%v, len=%d, cap=%d\n", x, len(x), cap(x))` the result will be `x=[], len=0, cap=0`.

Step 2

When we use the append function, we add a new element (one or more) to the slice:

```
x = append(x, 1)
```

Since our slice `x` was empty, function `append(x, 1)` will create a new underlying array (slice expansion) with a length of 1 and a capacity of 1 (applicable for version 1.18 and beyond, in some other versions of Go, append may assign an initial capacity of 2).

If we print the result we will get `x=[1], len=1, cap=1`. Now the slice has a specific area in memory where first element 1 is stored. The length has become 1, and the capacity is 1.

Step 3

Appending second element:

```
x = append(x, 2)
```

When we use the `append` function to add a new element, it internally checks the slice for capacity: Whether a new element can be added to the previously allocated memory area using the inequality `len(x) + 1 <= cap(x)`.

Since in our case the new slice length will become larger than the capacity of underlying array, a slice extension will be performed. A new underlying array of larger size will be created and the values from the old memory area will be copied to the new one with the capacity: `cap = current cap * 2` (this rule for increasing capacity applies to a slice length less than 1024. Beyond that, the slice will not be increased by 100% (doubled), but by 25%).

After this, in the new memory area, we can add an element to the array, and we get: `x=[1, 2], len=2, cap=2`.

Step 4

```
x = append(x, 3)
```

Just like last time, the current length equals the capacity of our slice, so when adding a new element, our slice will be copied to a new memory area with a capacity of `2 * 2 = 4` and the result is `x=[1,2,3], len=3, cap=4`.

Here, the capacity has increased by 2, and instead of 2, it became 4.

Step 5

```
y:=x
```

We create a new variable `y`, which equals our slice `x`: `y=[1,2,3], len=3, cap=4`.

Step 6

We add a new element to the slice x:

```
x = append(x, 4)
```

The slice `x` had a length `len = 3`. This means the new element is added to the 4th position. And now the length will be 4, with the same capacity 4.

New length does not exceed the permissible capacity, there will be no extension of the slice, and our pointer in the slice will still look at the same memory area.

Get Nina Pakshina's stories in your inbox
Join Medium for free to get updates from this writer.
Enter your email Subscribe
☑ Remember me for faster sign in

The result of this step is: `x=[1,2,3,4], len=4, cap=4`.

Step 7

```
y = append(y, 5)
```

It is important to note here that we are adding a new element to `y` slice. We created `y` when `x` was equal to `x=[1,2,3], len=3, cap=4`.

In `y` we keep a reference to the first element of the slice, our array length is 3 and capacity is 4. Therefore, the new element 5 is inserted into the 4th position just as well, and there will be no extension of the slice. This means that the value 4, which we saved in step 6 in `x`, will be overwritten by value 5 at this step.

Result: `y=[1,2,3,5], len=4, cap=4`

Step 8

```
x[0] = 0
```

We change the value of the first element in `x`.

In the previous steps with append, we did not have slice expansion. Therefore, both `x` and `y` point to the same element at index 0. The operation `x[0] = 0` will put a new value 0 in the element in both array `x` and array `y`.

So the final result for lines `fmt.Println(x)` and `fmt.Println(y)` will be the same: `[0, 2, 3, 5]` and `[0, 2, 3, 5]`.

For convenience, the step-by-step change of `len` and `cap` in the code:

```
func main() {
  var x []int      // x=[], len=0, cap=0
  x = append(x, 1) // x=[1], len=1, cap=1
  x = append(x, 2) // x=[1, 2], len=2, cap=2
  x = append(x, 3) // x=[1, 2, 3], len=3, cap=4
  y := x           // y=[1, 2, 3], len=3, cap=4
  x = append(x, 4) // x=[1, 2, 3, 4], len=4, cap=4
  y = append(y, 5) // x=[1, 2, 3, 5], len=4, cap=4
  x[0] = 0         // x=[0, 2, 3, 5], len=4, cap=4

  fmt.Println(x)   // x=[0, 2, 3, 5], len=4, cap=4
  fmt.Println(y)   // y=[0, 2, 3, 5], len=4, cap=4
}
```

Example 2: Slice expansion

Let's look at another question:

```
func main() {
  x := []int{1,2,3,4}
  y := x
  x = append(x, 5)
  y = append(y, 6)
  x[0] = 0
  fmt.Println(x)
  fmt.Println(y)
}
```

What will `fmt.Println(x)` and `fmt.Println(y)` output here?

Here is the answer right away: `[0,2,3,4,5]` for `x` and `[1,2,3,4,6]` for `y`.

Why in this case `x[0] = 0` replaced the first element only in the slice `x`, but not in `y`? And the last element is also different for `x` and `y`?

The fact is that at the moment of adding a new element to `x` and `y`, the slice had a length of 4 and a capacity of 4.

During the `x = append(x, 5)` the slice expanded into a new area of memory, where the capacity now became equal to 8.

Now `x` and `y` stopped referring to the same first element of the slice, and ended up in different areas of memory. Similarly, for this reason adding the value 6 to the slice y did not affect the x slice at all.

For convenience, the step-by-step change of `len` and `cap` in the code:

```
func main() {
  x := []int{1,2,3,4} // x=[1,2,3,4], len=4, cap=4
  y := x              // y=[1,2,3,4], len=4, cap=4
  x = append(x, 5)    // x=[1,2,3,4,5], len=5, cap=8
  y = append(y, 6)    // y=[1,2,3,4,6], len=5, cap=8
  x[0] = 0            // x=[0,2,3,4,5], len=5, cap=8

  fmt.Println(x)      // x=[0,2,3,4,5], len=5, cap=8
  fmt.Println(y)      // y=[1,2,3,4,6], len=5, cap=8
}
```

Example 3. Subslices

Let's consider another example:

```
package main

import "fmt"

func main() {
  x := []int{1, 2, 3, 4, 5}
  x = append(x, 6)
  x = append(x, 7)
  a := x[4:]
  y := alterSlice(a)
  fmt.Println(x)
  fmt.Println(y)
}

func alterSlice(a []int) []int {
  a[0] = 10
  a = append(a, 11)
  return a
}
```

What will `fmt.Println(x)` and `fmt.Println(y)` print?

Let's go step by step through our code. When creating `x` we initialize the slice with initial values. At this point, the slice will be preallocated and the length and capacity of the slice will be equal to 5:

```
x := []int{1, 2, 3, 4, 5} // x = [1,2,3,4,5], len=5, cap=5
```

Next, when we add a new element to the slice, which has len = cap, slice expansion will occur, the capacity will double to 10:

```
x = append(x, 6) // x = [1,2,3,4,5,6], len=6, cap=10
x = append(x, 7) // x = [1,2,3,4,5,6,7], len=7, cap=10
```

Next, we create a slice from the 4th element to the end of our slice.

```
a := x[4:]
```

The length of the slice in our case will be equal to the length of the original slice minus the first index of the slice: `len(a) = 7 — 4 = 3`.

Meanwhile, the capacity of the slice is calculated as the capacity of the original slice minus the first index of the slice: `cap(a) = 10 — 4 = 6`.

Thus, we get a slice: `a = [5,6,7], len=3, cap=6.`

The function `alterSlice` changes the 0 index of our slice a, which points to the 4th index of the slice `x.`

Then, the value 11 is added to a, and since the inequality `len(a) + 1 < cap(a)` holds true there will be no relocation of data. And as a result, we get:

`[1,2,3,4,10,6,7]` for `x` and `[10, 6, 7, 11]` for `y`.

Code with step-by-step change:

```
import "fmt"
func main() {
  x := []int{1, 2, 3, 4, 5} // [1,2,3,4,5], len=5, cap=5
  x = append(x, 6)          // [1,2,3,4,5,6], len=6, cap=10
  x = append(x, 7)          // [1,2,3,4,5,6, 7], len=7, cap=10
  a := x[4:]                // [5,6,7], len=3, cap=6
  y := alterSlice(a)        // [10, 6, 7, 11], len=4, cap=6

  fmt.Println(x)            // [1,2,3,4,10,6,7], len=7, cap=10
  fmt.Println(y)            // [10, 6, 7, 11]
}
func alterSlice(a []int) []int {
  a[0] = 10                 // [10, 6, 7], len=3, cap=6
  a = append(a, 11)         // [10, 6, 7, 11], len=4, cap=6
  return a
}
```

Bonus question: What will happen if you try to print `fmt.Println(x[0:8])` at the end of the code? You can write your answers in the comments.

Summary to remember in order to crack these interview questions

1. A slice is a reference data type. Inside there is a pointer to the first element of the slice. This factor is what determines how certain operations, even when performed on copies of the slice, can affect the original slice.
2. A slice has a length, which describes the number of elements currently stored in the slice, and a capacity, which indicates how many elements can be added to this memory area.
3. If the inequality `len(x) + 1 <= cap(x)` is not met when adding a new element, the slice expands into a new area of memory, and capacity doubles (until it reaches the size of 1024, after which they increase by 25% with each expansion).
4. When you pass a slice as an argument to a function as a copy (not via a pointer), you should remember that the slice contains a pointer to the first element, which allows for modifications to the original slice.
5. The length and capacity values are passed by copy. If you pass a slice to a function and then the same slice is modified elsewhere in the code (e.g., by adding a new element), it will not affect the length and capacity of the copied slice within the function.

I wish you success in handling slices during your interviews!

Golang + Golang Tutorial + Interview + Programming + Programming Languages +

439 6

Written by Nina Pakshina
957 followers · 3 following
Follow

Go developer in an online retail company. LinkedIn: https://www.linkedin.com/in/nina-pakshina-228156114/ Buy me a coffee: https://www.buymeacoffee.com/ninucium

Responses (6)

Write a response

What are your thoughts?

Ramuece
Apr 17, 2024

What a awesome content Nina :) Great going!

1 Reply

Otabek Muzropov
Sep 11, 2023 (edited)

Hi Nina Pakshina. Thanks for this article. I checked "this rule for increasing capacity applies to a slice length less than 1024. Beyond that, the slice will not be increased by 100% (doubled), but by 25%)" this part. But In my opinion this is not… more

1 Reply

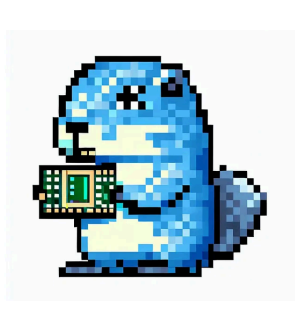
Rgromozeka
Jul 22, 2025

вроде бы есть оптимизация int32 int64

Reply

See all responses

More from Nina Pakshina



Nina Pakshina · Sep 19, 2023

Parallelism and Concurrency in Go: How It Works in Real Computing…

This article is for those who are familiar with the basic elements of concurrency but want…

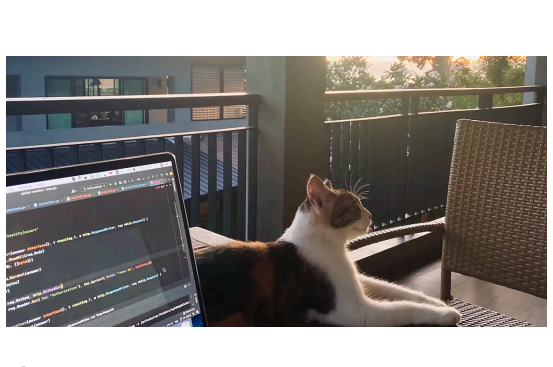
303 4

Nina Pakshina · Oct 3, 2023

Parallelism and Concurrency in Go: How It Works in Real Computing…

How multitasking is structured at higher levels: Operating System Level, Programmin…

143 1

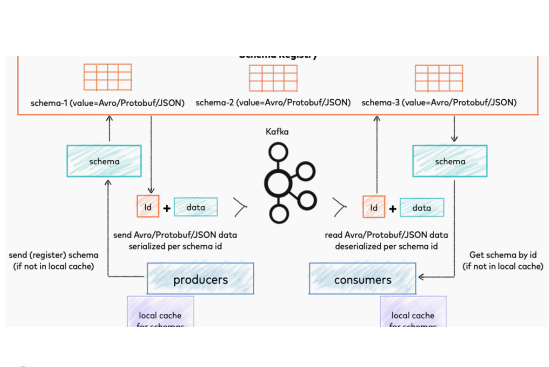


Nina Pakshina · May 16, 2023

Golang Concurrency Patterns: For-Select-Done, Errgroup and Worke…

Advanced concurrency patterns in Go that can be really handy: for-select-done,…

492 6



Nina Pakshina · Feb 26, 2023

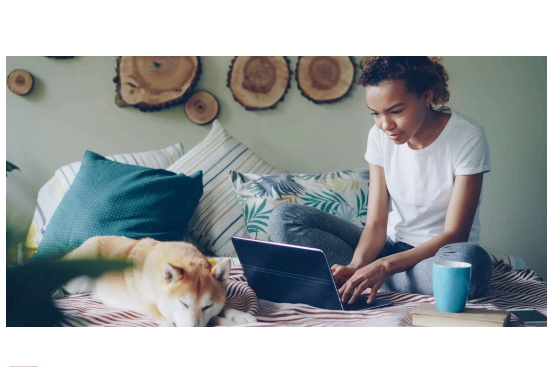
Using Kafka with Schema Registry and Protobuf. Part 1.

My name is Nina and I work as a Golang Backend Software Engineer at a large retail…

99 6

See all from Nina Pakshina

Recommended from Medium

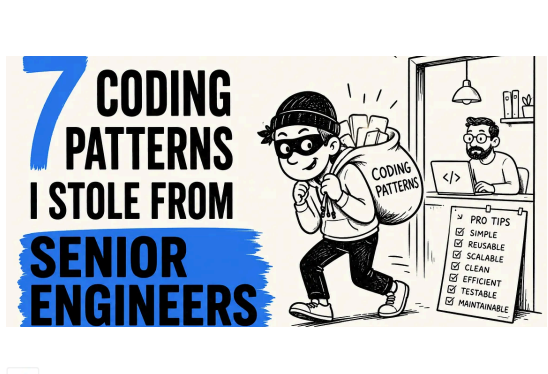


In The Sunday Journal by Rafia Naseem · Jul 18

10 Websites Better Than Another Hour of Doomscrolling

Here's to the rabbit holes worth taking!

9.6K 226 114



In Skill Stuff by CodeByUmar · Jun 8

7 Coding Patterns I Stole From Senior Engineers

Most developers do not become better because they learn more syntax. They…

4K 76 85

Vijeta · Mar 2

Blinkit Interview: SDE Intern

How I utilized my Blinkit Interview for Projects

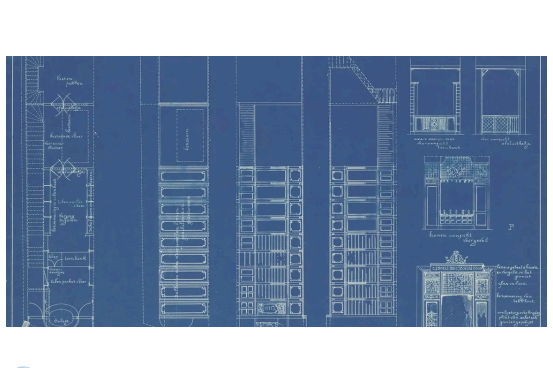
1

In Bootcamp by Priyank Chheda · Feb 21

Understanding the Facade Design Pattern in Go: A Practical Guide

Simplifying Complex Systems With a Clean, Unified API

3

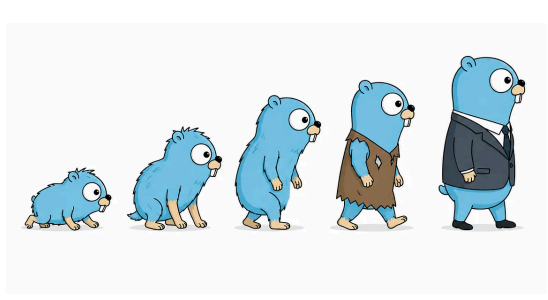


Pascal Allen · May 20

Structuring Go Projects With Clean Architecture

In this publication, I will walk through how I structure my Go projects. This is a high-level…

10



In Towards Dev by Cheikh seck · 2d ago

The Evolution of Go Generics: A Retrospective from 1.18 to 1.27

From type parameters to generic methods: how Go's type system matured across three…

1

See more recommendations